START HERE: the whole project, end to end, in simple words

Everyone reads this first. After this, each person reads their own doc (`1_gcs.md`, `2_drone.md`, `3_session.md`, `4_attacks.md`).

Reading time: about 30 minutes. Running everything: about 5 minutes.

Part 1: The project in 30 seconds

Title 6: *Design a secure protocol for authentication between drone-to-drone communication.*

Two drones are flying. They want to talk over radio, for example "I'm at 120 m, hold position". The problem is that **anyone with an antenna can listen to, record, change, or fake radio messages.**

So we designed a set of rules (a **protocol**) so that:

1. Each drone can **prove who it is** to the other drone (authentication).
2. They **agree on a secret key** that nobody else knows.
3. They use that key to send messages that are **secret** (no one can read them) and **tamper-proof** (no one can change them).
4. **Old messages can't be replayed**, and **fake or captured drones are rejected.**

Then we **attacked our own protocol 13 different ways** to show that every attack fails.

Everything is in Python, using one library called `cryptography`.

Part 2: The problem, as a story

Imagine two soldiers, **Alpha** and **Bravo**, who have never met. They have to coordinate over walkie-talkies, and the enemy is listening on the same channel.

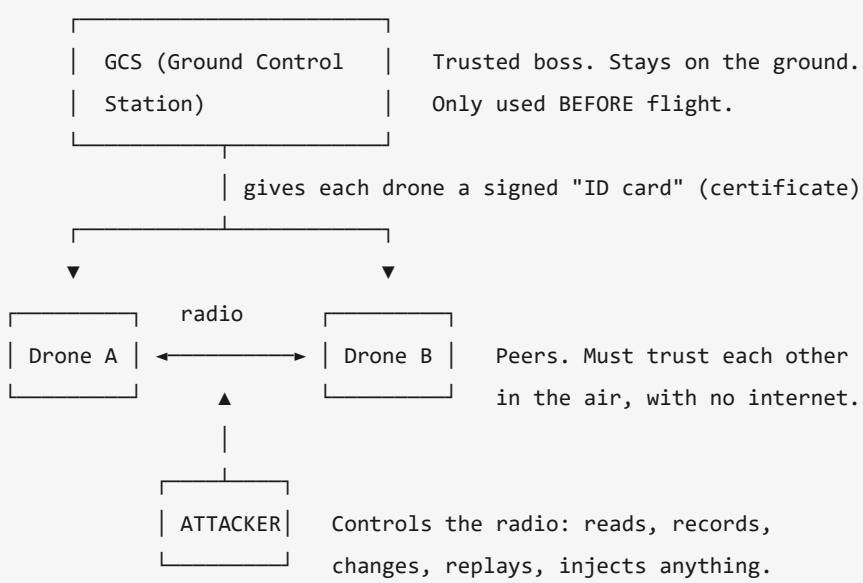
What can go wrong?

Problem	Example	What we need
Eavesdropping	Enemy hears "we're at grid 17.98, 79.53"	**Confidentiality**: messages must be unreadable
Fake identity	Enemy says "Hi, I'm Alpha"	**Authentication**: prove who you are
Changed message	Enemy changes "hold position" to "land now"	**Integrity**: detect any change
Replay	Enemy records "land now" and plays it again tomorrow	**Freshness**: old messages must be rejected

Problem	Example	What we need
Man in the middle	Enemy sits between them, pretending to be Bravo to Alpha and Alpha to Bravo	**Mutual authentication + key agreement**
Captured soldier	Enemy captures Alpha's radio	**Revocation**: cancel Alpha's identity

Our drones have exactly these problems. Our protocol solves every row.

Part 3: The three characters



- **GCS**: like a passport office. It issues ID cards (certificates) to drones before they fly.
- **Drones**: once in the air, they can't call the GCS. They must check each other on their own.
- **Attacker**: we assume the attacker controls the whole radio channel. This is called the **Dolev-Yao model**. The only thing it can't do is break the maths or steal private keys from inside an honest drone.

Part 4: The building blocks, one analogy each

You need these 8 ideas. Everything else is just combining them.

4.1 Key pair (private key + public key)

- **Private key:** a secret only you have, like your **signature hand**.
- **Public key:** something everyone can know, like a **sample of your signature** that others use to check it's really yours.

4.2 Digital signature (we use **Ed25519**)

- You "sign" a message with your **private** key. Anyone can check it with your **public** key.
- If even one letter of the message changes, the check fails.
- Nobody can fake your signature without your private key.

- **Used for:** proving who sent a message, and that it wasn't changed.

4.3 Certificate

- A small "ID card": `{drone ID, drone's public key, expiry time}` **signed by the GCS**.
- Every drone knows the GCS's public key, so it can check any ID card offline.
- **Used for:** "The GCS says this public key really belongs to Drone A."

4.4 Diffie-Hellman key exchange (we use **X25519**)

- The paint analogy: both sides start with a common colour. Each adds a **secret colour** and sends the mix. Each then adds their secret colour to the *other's* mix, and both end up with the **same final colour**. Someone watching only saw the mixes and can't separate the colours to get the final one.
- In maths: A sends `a·G` and B sends `b·G`. Both compute `a·b·G`. An eavesdropper can't.
- **Used for:** creating a shared secret over an open channel.
- **"Ephemeral"** means we make **new** secret colours for every conversation and throw them away afterwards, so recording today's traffic and stealing a drone tomorrow still won't reveal today's key. This is called **forward secrecy**.

4.5 Hash (SHA-256)

- A "fingerprint" of data. Same input gives the same fingerprint. Any change gives a completely different fingerprint. You can't go backwards.

4.6 HMAC

- A fingerprint that needs a **secret key** to make. Only people with the key can produce the right HMAC.
- **Used for:** message 3, so Drone A proves "I have the same key as you".

4.7 HKDF (key derivation)

- The Diffie-Hellman result is "raw material", not a clean key. HKDF is a **key factory**: raw secret in, clean, perfectly random keys out.
- We get **two** keys from it: one for encryption (`K_enc`) and one for HMAC (`K_mac`).

4.8 AES-256-GCM (authenticated encryption)

- **AES** is a lock that scrambles a message so only the key holder can unscramble it.
- **GCM** adds a **tamper seal**: if anyone changes even one bit, unscrambling fails loudly.
- It needs a **nonce** ("number used once") per message. Never reuse one, or security breaks. We use a counter, 1, 2, 3..., so it never repeats.

Plus two freshness tools

- **Nonce in the handshake:** a random 16-byte number. The receiver remembers nonces it has seen, so a repeated one means replay.
- **Timestamp:** "sent at 10:05:03". If it's more than 30 seconds old, it's rejected. Drones have GPS, so their clocks are accurate.

Part 5: The protocol, step by step

Phase 0: Registration (on the ground, before flight). File: `gcs.py`

```
Drone A makes its own key pair (private key stays inside the drone).
Drone A ──(public key)──► GCS
GCS ──(certificate = Sign_GCS{ID "A", A's public key, expiry})──► Drone A
GCS also loads its OWN public key into every drone.
```

Every drone now has: its private key, its certificate, and the GCS public key.

Phase 1: The handshake in the air (3 messages). File: `drone.py`

```
Drone A                                                  Drone B
   │                                                        │
   │ M1: my certificate, my DH public key, nonce,           │
   │     timestamp, all SIGNED by me                        │
   │────────────────────────────────────────────────────────►│
   │                                  B CHECKS:             │
   │                                  1. Is it addressed to me?
   │                                  2. Is the timestamp fresh?
   │                                  3. Is the nonce new?
   │                                  4. Is the cert signed by OUR GCS,
   │                                     not expired, not revoked?
   │                                  5. Is A's signature valid?
   │                                                        │
   │ M2: B's certificate, B's DH public key, nonce,         │
   │     timestamp, SIGNED by B over (M1 + M2)              │
   │◄────────────────────────────────────────────────────────│
   │ A CHECKS the same 5 things.                            │
   │                                                        │
   │ Now BOTH compute the same Diffie-Hellman secret,       │
   │ and turn it into K_enc + K_mac (HKDF).                 │
   │                                                        │
   │ M3: HMAC(K_mac, everything said so far)                │
   │────────────────────────────────────────────────────────►│
   │                                  B computes the same HMAC.
   │                                  Match = A really has the key.
   │                                                        │
   │              SESSION READY: both hold K_enc            │
```

Why does B sign M1 + M2 together? It glues B's answer to *this exact* question. The attacker can't reuse an old answer, and can't change A's DH key in M1 without B's signature breaking.

Phase 2: Secure talk. File: `session.py`

```
A: seal("hold at 120 m")  → {seq: 1, ct: 8f3a91...}   (AES-GCM encrypted + tamper seal)
B: open(...)              → checks seq is new, checks seal, decrypts → "hold at 120 m"
```

- The **sequence number** (1, 2, 3...) blocks replays and makes each nonce unique.
- **Changed one bit?** The seal breaks and the packet is rejected.

Phase 3: Attack testing. File: `attacks.py`

We play the attacker and try 13 attacks. Each attack must be rejected for the **right reason**.

Part 6: Why each piece exists (remove it and see what breaks)

This is the most important table for the viva. Every piece is there for a reason.

If we removed...	This attack would work	Proven by test
Certificate check (GCS signature)	Rogue drone with a fake ID card gets in	`test_impersonation_fake_cert`
Message signature	Attacker swaps DH keys (MITM), or uses a stolen certificate	`test_mitm_*`, `test_stolen_cert_without_key`
Nonce cache	Recorded M1 replayed	`test_replay_handshake`
Timestamp	Old handshake replayed later	`test_stale_timestamp`
Expiry in the cert	Old certificates work forever	`test_expired_cert`
Revocation list	Captured drone still accepted	`test_revoked_drone`
"to" field check	Message for B accepted by C	`test_misdirected_message`
M3 confirmation	B can't be sure A really has the key	`test_forged_key_confirmation`
AES-GCM tamper seal	Attacker flips bits of commands	`test_tampered_ciphertext`
Sequence number	"land now" replayed	`test_replay_data_packet`
Ephemeral (fresh) DH keys	Same key every session, so no forward secrecy	`test_fresh_keys_per_session`

Part 7: How the code is organised

7.1 Four files, one per person

File	Person	One-line job
`gcs.py`	1	The ID-card office: issue, check, cancel certificates
`drone.py`	2	The drone and the 3-message handshake

File	Person	One-line job
`session.py`	3	Key factory + encrypted channel
`attacks.py`	4	The attacker: 13 attacks + 1 normal run

7.2 Who uses whom

```
gcs.py          ← the base; uses none of our other files
  ▲  ▲
  │  └── session.py      takes: AuthError, ser
  │         ▲
  │         │  takes: derive_keys, make_tag, check_tag, Session
drone.py ─────┘            takes from gcs: GCS, verify_certificate, verify_sig, pub_hex
  ▲
  │  takes: Drone, handshake
attacks.py
```

`AuthError` (defined in `gcs.py`) is the "REJECTED!" signal used by every file. Whenever any check fails, it raises `AuthError("reason")`.

7.3 Following one complete run through the functions

This is what happens, function by function, when two drones talk:

```
gcs = GCS()                                  gcs.py      GCS makes its signing key
a = Drone("A", gcs)                          drone.py    A makes its key pair
   └─ gcs.register("A", A's public key)      gcs.py      GCS signs A's certificate
b = Drone("B", gcs)                          (same for B)

m1 = a.initiate("B")                         drone.py    A: new DH key, nonce, time, sign → M1

m2 = b.respond(m1)                           drone.py
   ├─ b._check(m1)                           drone.py    to-me? fresh? new nonce?
   │    └─ verify_certificate(...)           gcs.py      GCS-signed? expired? ID? revoked?
   │    └─ verify_sig(...)                   gcs.py      A's signature OK?
   └─ B: new DH key, nonce, time, sign(M1+M2) → M2

m3, session_a = a.finish(m2)                 drone.py
   ├─ a._check(m2)                           (same 5 checks on B)
   ├─ dh(...)                                drone.py    DH shared secret
   ├─ derive_keys(...)                       session.py  HKDF → K_enc, K_mac
   ├─ make_tag(K_mac, transcript)            session.py  HMAC → M3
   └─ Session(K_enc, role 0)                 session.py  A's secure channel

session_b = b.confirm(m3)                    drone.py
   ├─ dh(...), derive_keys(...)              B computes the same keys
   ├─ check_tag(...)                         session.py  M3 correct?
   └─ Session(K_enc, role 1)                 B's secure channel

packet = session_a.seal(b"hold")             session.py  encrypt + seal
session_b.open(packet)                       session.py  check seq, check seal, decrypt
```

If you can follow this trace, you understand the whole project.

Part 8: What to run, how to run it, what will happen

8.1 One-time setup (Windows)

Open a terminal in the project folder (`DSP`) and run:

```
python --version                  # needs 3.10 or newer (we tested 3.11.9)
pip install -r requirements.txt   # installs the "cryptography" library
pip install pytest                # optional, only for "pytest attacks.py"
```

8.2 Run each file (this is the presentation order)

> Every run uses **new random keys**, so the long hex strings will differ from the ones shown here. That's expected and correct.

Person 1: `python gcs.py`

```
1. GCS issued certificate: {'id': 'DRONE-ALPHA', 'pk': '572be41f...', 'exp': 1793814277}
2. Verified -> accepted, pk = 572be41fdd02630e ...
3. Attacker edits expiry          -> certificate not signed by GCS
4. Cert from a fake GCS           -> certificate not signed by GCS
5. Drone captured, GCS revokes it -> drone revoked
```

What each line means:

1. The GCS created an ID card for DRONE-ALPHA. `pk` is the drone's public key and `exp` is the expiry (Unix time, 1 hour ahead).
2. Another drone checked the card and it's genuine.
3. The attacker changed the expiry to make the card last longer. The GCS signature no longer matches, so it's **rejected**.
4. The attacker ran its own fake GCS. That card isn't signed by *our* GCS, so it's **rejected**.
5. The drone was captured and the GCS revoked it. Even a fresh, genuine card is now **rejected**.

Person 2: `python drone.py`

```
M1 ALPHA -> BRAVO : ['type', 'from', 'to', 'cert', 'eph', 'n', 'ts', 'sig']
M2 BRAVO -> ALPHA : ['type', 'from', 'to', 'cert', 'eph', 'n', 'ts', 'sig']  (BRAVO checked ALPHA's
cert + signature)
M3 ALPHA -> BRAVO : ['type', 'from', 'to', 'tag']  (ALPHA checked BRAVO's cert + signature)
BRAVO checked the confirmation tag

Same session key on both drones: True
BRAVO reads: hold position at 120 m
```

What it means:

- These are the **fields** inside each message: `cert` = ID card, `eph` = Diffie-Hellman public key, `n` = nonce, `ts` = timestamp, `sig` = signature, `tag` = HMAC confirmation.
- Each side checked the other's ID card and signature.
- `Same session key on both drones: True` means **both drones computed the same secret key without ever sending it.**
- The last line is a real encrypted message, decrypted correctly by Bravo.

Person 3: `python session.py`

```
1. ALPHA sends (ciphertext): 990fe968dc5deb5907c2931c3c280308dc4c8cd4 ...
2. BRAVO decrypts          : {"lat":17.98,"lon":79.53,"alt":120}
3. Attacker replays packet  -> replayed/old message
4. Attacker flips one bit   -> integrity check failed (tampered ciphertext)
```

What it means:

1. This is what the attacker sees on the radio: random-looking bytes.
2. Bravo, with the key, gets the real GPS message back.
3. The attacker re-sent the same packet. The sequence number is old, so it's **rejected**.
4. The attacker changed 1 bit. The AES-GCM tamper seal broke, so it's **rejected**.

Person 4: `python attacks.py`

```
PASS  test_honest_run               Baseline: two honest drones agree on a key and talk.
PASS  test_replay_handshake         Replay of a recorded M1 -> nonce cache.
PASS  test_replay_data_packet       Replay of a recorded 'land now' packet -> sequence counter.
PASS  test_stale_timestamp          Replay of an old handshake -> timestamp window.
PASS  test_mitm_swaps_dh_key        MITM swaps A's DH key in M1 -> A's signature no longer matches.
PASS  test_mitm_on_response         MITM swaps B's DH key in M2 -> B's signature no longer matches.
PASS  test_impersonation_fake_cert  Rogue drone with a cert from the attacker's own GCS -> not signed
by our GCS.
PASS  test_stolen_cert_without_key  Attacker presents A's real cert but lacks A's private key -> bad
signature.
PASS  test_revoked_drone            Captured drone revoked by GCS -> refused (authorization).
PASS  test_expired_cert             Editing the cert's expiry -> breaks the GCS signature.
PASS  test_misdirected_message      Message for B delivered to C -> 'to' field check.
PASS  test_tampered_ciphertext      Flip one bit of a data packet -> AES-GCM tag fails.
PASS  test_forged_key_confirmation  Fake M3 with a made-up tag -> attacker doesn't know K_mac.
PASS  test_fresh_keys_per_session   Forward secrecy: new DH keys every time -> every session key is
different.

All 14 checks passed - every attack was blocked.
```

What it means: **PASS = the attack was blocked by the correct defence.** If any attack ever got through, you'd see `AssertionError: attack succeeded!` and the run would stop.

Optional, same tests through pytest: `pytest attacks.py` prints `14 passed`.

8.3 Break it yourself (the best way to really understand it)

Do this once tonight **on a copy of the folder**, not the real one. It shows the professor you understand *why* each check exists.

Experiment 1: remove the nonce check. In `drone.py`, inside `_check`, change

```
        if msg["n"] in self._seen_nonces:
```

to

```
        if False:
```

Run `pytest attacks.py`. You'll see:

```
FAILED attacks.py::test_replay_handshake - AssertionError: attack succeeded!
1 failed, 13 passed
```

→ Without the nonce cache, a recorded M1 can be replayed. (The timestamp alone doesn't stop a replay within 30 seconds.)

Experiment 2: remove the signature check. In `drone.py`, inside `_check`, change

```
        if not verify_sig(pk, msg["sig"], signed):
```

to

```
        if False:
```

Run `pytest attacks.py`. You'll see:

```
FAILED attacks.py::test_mitm_swaps_dh_key
FAILED attacks.py::test_mitm_on_response
FAILED attacks.py::test_stolen_cert_without_key
3 failed, 11 passed
```

→ Without signatures, a man-in-the-middle and someone with a stolen certificate both get in. **A certificate alone proves nothing. You must also prove you own the private key.**

(We ran both experiments ourselves, and these are the real results.) Undo the change afterwards, or just delete the copy.

Part 9: How this maps to the course (CS428 lecture plan)

Lecture topic	Where it appears in our project
Digital Signature Schemes	Certificates and handshake signatures (Ed25519)
Secure Authenticated Key exchange protocols	The whole M1/M2/M3 handshake
CDH/DDH, Discrete logarithm problem	X25519 Diffie-Hellman and Ed25519 security
SSL/TLS, session establishment, cipher suites	Our handshake has the same shape as TLS. Our "cipher suite" = X25519 + Ed25519 + HKDF-SHA256 + AES-256-GCM
Authenticated Encryption, CCA-Secure Encryption	AES-GCM in `session.py`
CPA-Security, Security for Multiple Encryptions	Unique nonce per packet (counter)
Modes of Operation, Block cipher	AES in GCM mode (= CTR + authentication)

Lecture topic	Where it appears in our project
Message Authentication Using Hash Functions, Merkle–Damgård	HMAC-SHA256 (M3)
Random-Oracle Model, CDH-Based KEM	HKDF turning the DH secret into keys
Types of Attacks (COA, KPA, CPA, CCA)	`attacks.py` and `docs/4_attacks.md`
Zero knowledge proofs	Ed25519 is a Schnorr-type signature, which comes from a ZK identification protocol
Application area: Vehicular Network	Drones = a flying vehicular network
Secret Sharing, Group Key agreement	Future work (see below)

Each person's doc explains their own rows in detail.

Part 10: Honest limitations (say these if asked; it earns marks)

1. **Only 2 drones at a time.** A whole swarm would need a **group key agreement** protocol. That's in the lecture plan and is our future work.
2. **The GCS key is a single point of trust.** If it's stolen, fake certificates become possible. Fix: **secret sharing / threshold signatures** (lecture topic).
3. **The revocation list is simulated.** In code, drones share the GCS's list directly. In reality, the GCS pushes a signed list before each mission.
4. **No formal proof.** Tests show these attacks fail. They don't prove *every* attack fails. Next step: verify with a tool like ProVerif or Tamarin.
5. **Out-of-order packets are dropped.** A real radio link would use a sliding window (like IPsec).
6. **Jamming or physical capture can't be stopped by crypto.** We only limit the damage, with revocation and short expiry.

Part 11: Ten questions everyone should be able to answer

1. **What does the protocol do in one line?** Two drones prove their identities using GCS-signed certificates, agree on a fresh key with Diffie-Hellman, and then talk with AES-GCM encryption.
2. **Why not just use a shared password for all drones?** One captured drone would expose the whole fleet. Per-drone keys plus revocation contain the damage.
3. **Why do drones need the GCS during flight?** They don't. Certificates are checked offline using the stored GCS public key.
4. **How is a man-in-the-middle stopped?** The DH keys are signed. Swapping one breaks the signature.
5. **How are replays stopped?** Nonces and timestamps in the handshake, sequence numbers in data packets.
6. **What is forward secrecy?** Fresh DH keys every session, deleted afterwards, so stealing a drone later doesn't reveal past conversations.
7. **Why 3 messages, not 2?** M3 proves A actually derived the same key (key confirmation).
8. **Why AES-GCM?** It gives confidentiality and integrity in one step (authenticated encryption, CCA-secure), and it's the TLS 1.3 standard.
9. **What does `AuthError` mean in the code?** "Rejected". Every failed security check raises it.

10. **What's the attack model?** Dolev-Yao: the attacker controls the network but can't break the crypto or read private keys.

Part 12: Who presents what

Order	Person	Opens	Runs	Doc to study
1	Person 1	`gcs.py`	`python gcs.py`	`docs/1_gcs.md`
2	Person 2	`drone.py`	`python drone.py`	`docs/2_drone.md`
3	Person 3	`session.py`	`python session.py`	`docs/3_session.md`
4	Person 4	`attacks.py`	`python attacks.py`	`docs/4_attacks.md`

Suggested opening (anyone): read Part 1 of this doc out loud and draw the Part 3 diagram on the board.
Suggested closing (Person 4): "All 14 checks pass, and every attack in our model is blocked. Our future work is group keys for swarms and threshold signing for the GCS."